Cat vs Dog Image Classifier

Transfer Learning with MobileNetV2

Deep Learning Semester Project Report

Author: Eshal Afzal Sial

Date: 30 October 2026

Live demo (Vercel)	https://cat-dog-classifier-web.vercel.app
GitHub repository	https://github.com/Eshal-Afzal-Sial/cat-dog-classifier
Dataset (Kaggle)	https://www.kaggle.com/datasets/tongpython/cat-and-dog

Abstract

This project builds an image classifier that decides whether a photograph shows a cat or a dog. It uses transfer learning: a MobileNetV2 network pre-trained on ImageNet is frozen and used as a feature extractor, and a small classification head is trained on about 8,000 labelled images. The model reaches **98.86% accuracy** on a held-out test set of 2,023 images it never saw during training, with a best validation accuracy of 99.38%. The trained model was exported to the ONNX format and deployed as a web application on Vercel, where it runs entirely inside the visitor's browser. Source code, weights and documentation are published in a public GitHub repository.

1. Problem Statement

Image classification is a core task in computer vision. The goal here is binary classification: given a colour photograph, predict whether the main subject is a **cat** or a **dog**. Although easy for humans, the task is non-trivial for software because animals vary in breed, pose, lighting, background and image quality. Hand-written rules do not work well, so a learned model is used.

The project has three objectives: (1) train an accurate classifier with a small, fast model; (2) make the trained weights reproducible and publicly available; and (3) deploy a working demonstration that anyone can try without installing software.

2. Dataset Description

The project uses a secondary dataset, **Cat and Dog** by tongpython, downloaded from Kaggle (about 228 MB, roughly 10,000 images). It is organised into a training set and a test set, each with one folder per class. The two classes are almost perfectly balanced, so no class weighting was needed.

Split	Cats	Dogs	Total
Training set (as provided)	4,000	4,005	8,005
used for training (80%)			6,404
used for validation (20%)			1,601
Test set (held out)	1,011	1,012	2,023

Data checks. A script (`src/check_data.py`) opened every image with Pillow to look for corrupt files. No corrupt images were found. The only unreadable entries were four hidden macOS `_DS_Store` files, which are not images and are ignored by the Keras loader. A directory listing of the test set also showed a few duplicate files (names ending in "(1)", with identical file sizes). This is a minor data-quality note: a handful of duplicates among 2,023 test images has a negligible effect on the reported accuracy.

Preprocessing. Images are resized to 224 x 224 pixels, the input size of MobileNetV2, and pixel values are scaled by the network's own preprocessing function. The split into training and validation subsets uses a fixed random seed (42) so the experiment is repeatable.

3. Model Architecture

The model uses transfer learning. MobileNetV2 was designed for mobile and embedded devices; it uses depthwise separable convolutions and inverted residual blocks to reach good accuracy with few parameters. Its ImageNet weights already encode general visual features (edges, textures, shapes, animal parts), so only a small new classifier needs to be learned.

Stage	Layer	Details
1	Data augmentation	Random horizontal flip, random rotation (0.1), random zoom (0.1). Active only during training.
2	Preprocessing	MobileNetV2 preprocess_input (scales pixels to the range -1 to 1).
3	MobileNetV2 base	Pre-trained on ImageNet, top removed, **frozen**. Output 7 x 7 x 1280.
4	Global Average Pooling	Reduces 7 x 7 x 1280 to a 1280-value vector.
5	Dropout	Rate 0.2, to reduce over-fitting.
6	Dense	1 unit, sigmoid activation. Output is the probability that the image is a dog.

Parameters	Count
Total	2,259,265 (8.62 MB)
Trainable (classification head only)	1,281 (5 KB)
Non-trainable (frozen MobileNetV2 base)	2,257,984

Class labels follow alphabetical folder order: **0 = cat, 1 = dog**. A prediction above 0.5 is reported as a dog, otherwise as a cat, and the confidence shown to the user is the probability of the winning class.

4. Training Details

Setting	Value
Framework	TensorFlow 2.21.0 / Keras 3.15.1, Python 3.11.9
Hardware	CPU only (TensorFlow has no native Windows GPU support)
Optimizer	Adam, learning rate 0.001
Loss function	Binary cross-entropy
Metric	Accuracy
Batch size	32 (201 steps per epoch)
Maximum epochs	8
Callbacks	ModelCheckpoint (saves the best model by validation accuracy); EarlyStopping (patience 3, restores best weights)
Time per epoch	about 160 to 175 seconds

Only the new classification head was trained; the MobileNetV2 base stayed frozen throughout. This keeps training fast on an ordinary laptop CPU and avoids over-fitting on a modest dataset. Training ran for all 8 epochs. Because early stopping was configured with `restore_best_weights`, the final saved model uses the weights from the epoch with the

highest validation accuracy (99.38%), not necessarily the last epoch.

5. Results

Metric	Value
Test accuracy (2,023 unseen images)	**98.86%**
Test loss	0.0344
Best validation accuracy	99.38%
Epoch 8 training accuracy / loss	97.89% / 0.0557
Epoch 8 validation accuracy / loss	99.31% / 0.0270

Interpretation. Test accuracy (98.86%) is close to validation accuracy (about 99%), which shows the model generalises well and is not over-fitted to the training images. Validation accuracy is slightly higher than training accuracy; this is normal here because data augmentation and dropout make the training pass harder than the validation pass. Accuracy and loss curves for every epoch are saved in `model/training_curves.png` in the repository.

Single-image checks. After training, the saved model was reloaded in a fresh Python process. A test cat image was predicted as *cat* (about 100% confidence) and a test dog image as *dog* (92.8% confidence).

Limitations. The model only knows two classes, so a photo of any other object will still be forced into "cat" or "dog". The web demo reduces this problem by showing "Not sure" when the confidence is below 70%. Accuracy was measured on one dataset and may be lower on very different photos (heavy occlusion, drawings, unusual angles).

6. Deployment

Model export. Vercel's serverless functions are too small to run TensorFlow, so the trained Keras model was converted to the ONNX format with `tf2onnx` (`src/convert_onnx.py`). The ONNX file is about 8.5 MB. The conversion was verified (`src/verify_onnx.py`): for a test image, Keras output 0.927832 and ONNX output 0.927833, a difference of about 0.0000005.

Web application. A single static page (`web/index.html`) loads ONNX Runtime Web and the model file, resizes the chosen image to 224 x 224 in the browser, and displays the prediction and confidence. Because inference happens client-side, uploaded photos never leave the user's device. The page also lists the dataset, code and model details, the results, and links to the GitHub repository, report, presentation and video.

Hosting. The repository is connected to Vercel with the root directory set to `web`, and every push to the main branch redeploys the site. The model file is publicly downloadable from the site, so it can also be fetched and tested directly.

Item	Link
Live demo	https://cat-dog-classifier-web.vercel.app
Model file (ONNX)	https://cat-dog-classifier-web.vercel.app/cat_dog_model.onnx
GitHub repository	https://github.com/Eshal-Afzal-Sial/cat-dog-classifier

7. Repository Contents

Path	Purpose
model/cat_dog_model.keras	Full trained Keras model
model/cat_dog.weights.h5	Trained weights
model/cat_dog_model.onnx	ONNX export used by the web demo
model/training_curves.png	Accuracy and loss curves
src/check_data.py	Counts images and checks for corrupt files
src/train.py	Loads data, builds, trains and evaluates the model
src/predict.py	Predicts the class of one image
src/convert_onnx.py, src/verify_onnx.py	ONNX export and check
web/index.html, web/cat_dog_model.onnx	Deployed web application
requirements.txt, README.md	Dependencies and documentation

To reproduce the results: install the packages in `requirements.txt`, download the dataset from Kaggle into `dataset/data/`, run `python src/train.py`, then `python src/predict.py path/to/image.jpg`. The dataset images themselves are not stored in the repository because of their size.

8. Conclusion and Future Work

The project shows that transfer learning gives high accuracy with very little training cost: only 1,281 parameters were trained, yet the model classifies 98.86% of unseen test images correctly. Converting to ONNX made it possible to deploy a real-time demo on a free static hosting platform.

Possible improvements include fine-tuning the top layers of MobileNetV2, adding an "other" class so unrelated images are rejected, adding Grad-CAM heat maps to show which image regions drive each prediction, comparing against a simple CNN trained from scratch, and testing on photos from outside the dataset.

References

- Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., Chen, L.-C. (2018). MobileNetV2: Inverted Residuals and Linear Bottlenecks. CVPR.
- Deng, J. et al. (2009). ImageNet: A large-scale hierarchical image database. CVPR.
- Kaggle dataset: Cat and Dog by tongpython, https://www.kaggle.com/datasets/tongpython/cat-and-dog
- TensorFlow / Keras documentation, https://www.tensorflow.org and https://keras.io
- ONNX and ONNX Runtime documentation, https://onnxruntime.ai